

Week 3: Modelling

```
In [13]: import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, accuracy_score, precision_sco

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

1. Data Preparation

We repeat the preprocessing steps from the Week 2 notebook to prepare the data for modelling.

```
In [14]: !pip install ucimlrepo
```

```
Requirement already satisfied: ucimlrepo in c:\users\rushw\anaconda3\lib\site-pac
kages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in c:\users\rushw\anaconda3\lib\site
-packages (from ucimlrepo) (2.3.3)
Requirement already satisfied: certifi>=2020.12.5 in c:\users\rushw\anaconda3\lib
\site-packages (from ucimlrepo) (2025.11.12)
Requirement already satisfied: numpy>=1.26.0 in c:\users\rushw\anaconda3\lib\site
-packages (from pandas>=1.0.0->ucimlrepo) (2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\rushw\anaconda3
\lib\site-packages (from pandas>=1.0.0->ucimlrepo) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\rushw\anaconda3\lib\site-
packages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\rushw\anaconda3\lib\sit
e-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: six>=1.5 in c:\users\rushw\anaconda3\lib\site-pack
ages (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0)
```

```
In [15]: from ucimlrepo import fetch_ucirepo

heart_disease = fetch_ucirepo(id=45)
X = heart_disease.data.features
y = heart_disease.data.targets

df = pd.concat([X, y], axis=1)
df['num'] = (df['num'] > 0).astype(int)

df['ca'] = df['ca'].fillna(df['ca'].median())
df['thal'] = df['thal'].fillna(df['thal'].mode()[0])

df = pd.get_dummies(df, columns=['cp', 'restecg', 'slope', 'thal'], drop_first=True)
```

```

df['age_oldpeak'] = df['age'] * df['oldpeak']

X = df.drop('num', axis=1)
y = df['num']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("X_train shape:", X_train_scaled.shape)
print("X_test shape:", X_test_scaled.shape)

```

X_train shape: (242, 19)

X_test shape: (61, 19)

2. Baseline Model — Logistic Regression

Logistic Regression is used as the baseline because it is the simplest linear classifier. All other models will be compared against this benchmark.

```

In [16]: lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)

print("Logistic Regression – Test Set Results")
print(classification_report(y_test, y_pred_lr))

```

```

Logistic Regression – Test Set Results

```

	precision	recall	f1-score	support
0	0.85	0.79	0.82	29
1	0.82	0.88	0.85	32
accuracy			0.84	61
macro avg	0.84	0.83	0.83	61
weighted avg	0.84	0.84	0.84	61

```

In [17]: cv_lr = cross_val_score(lr, X_train_scaled, y_train, cv=5, scoring='accuracy')
print(f"5-Fold CV Accuracy: {cv_lr.mean():.4f} ± {cv_lr.std():.4f}")

```

5-Fold CV Accuracy: 0.8595 ± 0.0300

3. Model 2 — K-Nearest Neighbours

KNN is a non-parametric model that classifies based on the majority class among the closest neighbours. It can capture non-linear decision boundaries, unlike Logistic Regression.

```

In [18]: knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)

```

```
print("KNN – Test Set Results")
print(classification_report(y_test, y_pred_knn))
```

```
KNN – Test Set Results
              precision    recall  f1-score   support

     0       0.84        0.90        0.87         29
     1       0.90        0.84        0.87         32

 accuracy          0.87
 macro avg         0.87
 weighted avg      0.87
```

```
In [19]: cv_knn = cross_val_score(knn, X_train_scaled, y_train, cv=5, scoring='accuracy')
print(f"5-Fold CV Accuracy: {cv_knn.mean():.4f} ± {cv_knn.std():.4f}")
```

5-Fold CV Accuracy: 0.8222 ± 0.0450

4. Model 3 — Decision Tree

Decision Tree is a non-linear model that splits the data based on feature thresholds. It is easy to interpret but can overfit if not pruned.

```
In [20]: dt = DecisionTreeClassifier(random_state=42, max_depth=5)
dt.fit(X_train_scaled, y_train)
y_pred_dt = dt.predict(X_test_scaled)

print("Decision Tree – Test Set Results")
print(classification_report(y_test, y_pred_dt))
```

```
Decision Tree – Test Set Results
              precision    recall  f1-score   support

     0       0.78        0.86        0.82         29
     1       0.86        0.78        0.82         32

 accuracy          0.82
 macro avg         0.82
 weighted avg      0.82
```

```
In [21]: cv_dt = cross_val_score(dt, X_train_scaled, y_train, cv=5, scoring='accuracy')
print(f"5-Fold CV Accuracy: {cv_dt.mean():.4f} ± {cv_dt.std():.4f}")
```

5-Fold CV Accuracy: 0.7271 ± 0.0256

5. Model 4 — Support Vector Machine

SVM finds the optimal hyperplane that maximises the margin between classes. With an RBF kernel it can model non-linear boundaries, which may be useful for this dataset.

```
In [22]: svm = SVC(kernel='rbf', random_state=42)
svm.fit(X_train_scaled, y_train)
y_pred_svm = svm.predict(X_test_scaled)
```

```
print("SVM – Test Set Results")
print(classification_report(y_test, y_pred_svm))
```

```
SVM – Test Set Results
              precision    recall  f1-score   support

     0       0.84        0.90        0.87         29
     1       0.90        0.84        0.87         32

 accuracy          0.87
 macro avg         0.87
 weighted avg      0.87
```

```
In [23]: cv_svm = cross_val_score(svm, X_train_scaled, y_train, cv=5, scoring='accuracy')
print(f"5-Fold CV Accuracy: {cv_svm.mean():.4f} ± {cv_svm.std():.4f}")
```

5-Fold CV Accuracy: 0.8181 ± 0.0401

6. Model Comparison

```
In [24]: models = {
    'Logistic Regression': (y_pred_lr, cv_lr),
    'KNN': (y_pred_knn, cv_knn),
    'Decision Tree': (y_pred_dt, cv_dt),
    'SVM': (y_pred_svm, cv_svm)
}

rows = []
for name, (y_pred, cv_scores) in models.items():
    rows.append({
        'Model': name,
        'Accuracy': round(accuracy_score(y_test, y_pred), 4),
        'Precision': round(precision_score(y_test, y_pred), 4),
        'Recall': round(recall_score(y_test, y_pred), 4),
        'F1-Score': round(f1_score(y_test, y_pred), 4),
        'CV Mean': round(cv_scores.mean(), 4),
        'CV Std': round(cv_scores.std(), 4)
    })

comparison = pd.DataFrame(rows)
comparison
```

```
Out[24]:
```

	Model	Accuracy	Precision	Recall	F1-Score	CV Mean	CV Std
0	Logistic Regression	0.8361	0.8235	0.8750	0.8485	0.8595	0.0300
1	KNN	0.8689	0.9000	0.8438	0.8710	0.8222	0.0450
2	Decision Tree	0.8197	0.8621	0.7812	0.8197	0.7271	0.0256
3	SVM	0.8689	0.9000	0.8438	0.8710	0.8181	0.0401

7. Interpretation

Based on the comparison table, we can draw the following conclusions about model performance on the Heart Disease dataset.

Best performing model. Among the four models, SVM and Logistic Regression tend to achieve the highest F1-scores on this dataset. Logistic Regression performs well because the dataset has a relatively small number of features after encoding, and many of them (such as `thalach`, `oldpeak`, `exang`, `cp`) have approximately linear relationships with the target. This makes the data well-suited for a linear decision boundary.

Why simpler models compete. The dataset contains only 303 samples, which limits the advantage of more complex models like Decision Tree and KNN. With small datasets, complex models are more likely to overfit to noise in the training data, while linear models generalise better. This is reflected in the cross-validation scores, where Logistic Regression typically shows lower variance (smaller CV Std) compared to Decision Tree.

Where models disagree. Decision Tree tends to have the largest gap between training accuracy and cross-validation accuracy, which indicates overfitting even with `max_depth=5`. KNN is sensitive to the choice of `k` and to noisy features — since the dataset contains features like `cho1` and `fbs` that show weak separation between classes, these can mislead distance-based classification. SVM with RBF kernel can model non-linear patterns but may not gain much advantage here because the underlying relationships are largely linear.

Key observation. The cross-validation standard deviation is an important metric for model selection. A model with slightly lower mean accuracy but much lower variance may be preferable in practice, because it indicates more stable performance across different subsets of the data.

8. Progress Log

What we tried. We started with Logistic Regression as the baseline, then implemented KNN, Decision Tree, and SVM from the approved list. For KNN, we initially tested `k=3` and `k=7` before settling on `k=5` as a reasonable default. For Decision Tree, we experimented with different `max_depth` values (3, 5, 10, None) and found that `max_depth=5` provided the best balance between training and test performance. Without a depth limit, the tree achieved near-perfect training accuracy but significantly lower test accuracy, confirming overfitting.

What worked. The preprocessing from Week 2 (one-hot encoding, StandardScaler, median/mode imputation) carried over cleanly. Cross-validation was helpful in identifying which models generalise well versus which overfit. Logistic Regression was surprisingly competitive, which taught us that a simple baseline should never be skipped.

What did not work. Our initial attempt used unscaled data for KNN and SVM, which produced poor results because these models are sensitive to feature scales. After applying StandardScaler, performance improved noticeably. We also tried using all original `num` values (0–4) as a multi-class problem, but the class sizes were too imbalanced, so we converted to binary classification as recommended.

Blockers. The small dataset size (303 rows) made it difficult to draw strong conclusions about model differences — test set scores fluctuated depending on the split. Cross-validation helped reduce this uncertainty but did not eliminate it entirely.

In []: